

Chapter 6, Section E: First-Class Functions (Functions as Values)

1. What “first-class” means

In JavaScript, a function is treated just like any other **value** — a number, a string, an object. This means a function can be:

- stored in a variable
- stored inside an object or array
- passed into another function as an argument
- returned out of another function

This is called being a **first-class citizen**. JavaScript doesn’t treat functions specially — it treats them like data.

2. Stored in a variable

```
const greet = function () {  
  console.log("Hi!");  
};  
  
greet(); // "Hi!"
```

You’re already doing this constantly — this is exactly what a function expression is.

3. Stored inside an object or array

```
const obj = {  
  sayHi: function () { console.log("Hi!"); }  
};  
obj.sayHi(); // "Hi!"  
  
const funcList = [  
  function () { console.log("one"); },  
  function () { console.log("two"); }  
];  
funcList[0](); // "one"  
funcList[1](); // "two"
```

A function sitting in an array slot is just a value in that slot, same as a number would be.

4. Passed into another function as an argument

```
function callTwice(fn) {  
  fn();  
  fn();  
}
```

```
function sayHello() {
  console.log("Hello!");
}

callTwice(sayHello);
// Hello!
// Hello!
```

Important detail: notice sayHello is passed **without parentheses** — callTwice(sayHello), not callTwice(sayHello()).

	Meaning
sayHello	The function itself — a value, not yet run
sayHello()	Runs the function immediately, and passes its result (whatever it returns)

```
callTwice(sayHello());
// sayHello() runs immediately ONCE, prints "Hello!"
// its return value (undefined) is passed to callTwice
// callTwice then tries to call undefined() twice -> TypeError
```

This mistake — adding () by accident — is one of the most common bugs beginners hit.

5. Returned out of another function

```
function makeMultiplier(factor) {
  return function (n) {
    return n * factor;
  };
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);

double(5); // 10
triple(5); // 15
```

makeMultiplier builds and **hands back** a new function. double and triple are both functions, created from the same “function factory,” each remembering a different factor.

(This is actually your first real look at a **closure** — covered properly in Section G. For now, just notice: a function can return another function.)

6. Why this matters — it enables two big patterns

Pattern	What it means	Covered in
Higher-order functions	A function that takes another function as an argument, or returns one	Section F
Callbacks	A function passed into another function, to be run	Section J

later

Functions like `array.map()`, `array.filter()`, `setTimeout()`, event listeners (`button.addEventListener()`) — all of these only work **because** functions are first-class. You're passing a function as a value into another function.

```
[1, 2, 3].map(function (n) {  
  return n * 2;  
});
```

Here, the anonymous function is just a **value** being handed to `.map()`. Nothing special syntactically — same idea as passing a number or string as an argument.

7. Quick contrast with other languages

Not every language works this way. In many older/lower-level languages, functions are a separate category from data and can't be passed around freely. JavaScript (along with Python, and others) chose to make functions first-class, which is why patterns like callbacks and higher-order functions are so natural in JS.

8. Summary

- Functions in JS are **values**, not a special separate thing.
 - They can be: stored in variables, stored in data structures, passed as arguments, returned from functions.
 - **Passing a function is not calling a function.** `fn` is the function itself; `fn()` runs it.
 - This property is the foundation for higher-order functions and callbacks — the next two sections.
-

9. Practice question

```
function outer() {  
  return function () {  
    console.log("inner ran");  
  };  
}  
  
const result = outer();  
console.log(typeof result);  
result();
```

Answer:

1. `outer()` runs and returns the inner function (does not run it yet) -> `result` now holds that function.
2. `typeof result` is "function", since `result` holds a function value.
3. `result()` calls the inner function, which prints "inner ran".

Output:

```
function  
inner ran
```